

RISC-V Packed SIMD Extension

P-Spec. Implementation

Version 0.92

Suchet Gopal

August 22, 2024

Contents

1	Introduction	2
2	Progress Made	3
3	Functional Units	4
3.1	The Multiplier	4
3.2	The Adder	7
3.2.1	Multi-level CLA Design	7
3.2.2	SWP CLA Design	8
3.3	Other Units	8
4	Overall Layout	9
5	Block Descriptions	9
5.1	ALU1	9
5.2	Multiplier	9
5.3	ALU Mux	10
5.4	ALU2	10
5.5	Accumulator and ALU3	10
5.6	Writeback Control	10
5.7	Miscellaneous Blocks	10
5.7.1	Shift Amount Generator	10
5.7.2	CLZ Blocks	11
5.7.3	CMP Blocks	11
6	Code Description	12

1 Introduction

This document describes the functional units developed for the P-Extension as well as the overall working of the module.

The files for this module were designed by Suchet Gopal (doc author) and Jaswant S. For any issues, drop me an email¹ and please also refer to this² thesis, which may contain some pointers.

This document is only meant to be used as a supplementary guide to the official p-spec documentation³.

The latest code version may be found here⁴.

¹gopal.suchet@gmail.com

²<https://repository.tudelft.nl/record/uuid:c4162ff8-9419-4434-852d-c1c3297df808>

³<https://github.com/riscv/riscv-p-spec/tree/master>

⁴<https://github.com/AtomicHydrogen/temp-riscvpsspec.git>

2 Progress Made

Instruction-wise progress can be found at this⁵ google sheet, which gives a comprehensive list of issues, bugs and working instructions as well as more internal details.

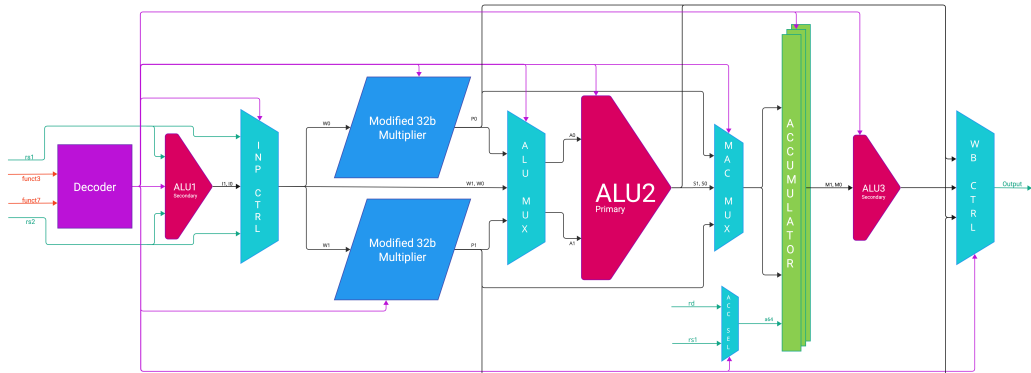


Figure 1: Final Datapath

The Datapath is subject to overhaul and change.

Around 195 instructions have been implemented as of July 2024. Future work requires major steps such as linking the multiplier and ALU2 serially, which would involve re-thinking saturation handling altogether. As mentioned earlier, the Accumulator as a whole needs to be implemented which would give rise to an entirely new set of problems to be dealt with - such as QSAT for more than two additions, and so on.

The instructions implemented so far consist of all the 'Basic' instructions and some of the fixed point instructions as classified in the previously mentioned thesis.

⁵<https://shorturl.at/qKFtB>

Section	Subsection	#Insns	Basic	MAC 8-bit	MAC 16-bit	MAC 32-bit	SAD 8-bit	Fixed-point	Non-SIMD 32- and 64-bit
1. SIMD Data Processing Instructions	1.1. 16-bit Addition & Subtraction Instructions	30	X						
	1.2. 8-bit Addition & Subtraction Instructions	10	X						
	1.3. 16-bit Shift Instructions	14	X						
	1.4. 8-bit Shift Instructions	14	X						
	1.5. 16-bit Compare Instructions	5	X						
	1.6. 8-bit Compare Instructions	5	X						
	1.7. 16-bit Multiply Instructions	6	X						
	1.8. 8-bit Multiply Instructions	6	X						
	1.9. 16-bit Misc Instructions	11	X						
	1.10. 8-bit Misc Instructions	11	X						
	1.11. 8-bit Unpacking Instructions	10	X						
2. Partial-SIMD Data Processing Instructions	2.1. 16-bit Packing Instructions	4	X			X			
	2.2. Most Significant Word "32x32" Multiply & Add Instructions	8			X				
	2.3. Most Significant Word "32x16" Multiply & Add Instructions	16			X				
	2.4. Signed 16-bit Multiply with 32-bit Add/Subtract Instructions	18			X				
	2.5. Signed 16-bit Multiply with 64-bit Add/Subtract Instructions	1			X				
	2.6.a. Miscellaneous Instructions a (32-bit)	5	X				X		
	2.6.b. Miscellaneous Instructions b (8-bit)	2							
	2.7. 8-bit Multiply with 32-bit Add Instructions	3							
3. 64-bit Profile Instructions	3.1. 64-bit Addition & Subtraction Instructions	10		X					
	3.2. 32-bit Multiply with 64-bit Add/Subtract Instructions	8				X			X
	3.3. Signed 16-bit Multiply with 64-bit Add/Subtract Instructions	10			X				
4. Non-SIMD Instructions	4.1. Q15 Saturation Instructions	7						X	
	4.2. Q31 Saturation Instructions	15						X	
	4.3. 32-bit Computation Instructions	9							X
	4.4. Overflow/Saturation Status Manipulation Instructions	2	X						
	4.5. Miscellaneous Instructions	11							X
5. RV64 Only Instructions	5.1. 32-bit Addition & Subtraction Instructions	30	X						
	5.2. 32-bit Shift Instructions	14	X						
	5.3. 32-bit Miscellaneous Instructions	5	X						
	5.4. Q15 Saturating Multiply Instructions	9						X	
	5.5. 32-bit Multiply Instructions	3	X						
	5.6. 32-bit Multiply & Add Instructions	3				X			
	5.7. 32-bit Parallel Multiply & Add Instructions	12				X			
	5.8. Non-SIMD 32-bit Shift Instructions	1							X
	5.9. 32-bit Packing Instructions	4	X						
Total Instructions		332	189	3	29	47	2	31	31

Figure 2: ISA Split-up

3 Functional Units

Only the functional units (and not the top level blocks) are described in this section.

The main functional units that use Subword Parallelism (SWP) include the multipliers, adders - contained within the ALU and the Accumulator. As it currently stands, the Accumulator and ALU3 are not yet implemented, and thereby neither are any MAC instructions.

3.1 The Multiplier

The Multiplier is the most area and delay intensive block in our datapath. The SWP multiplier is capable of performing 6 possible operations - SIMD 8b, 16b and 32b multiplication in both signed and unsigned formats. It handles signed multiplication using the Baugh-Wooley scheme.

The multiplier in essence has two serial blocks. The first is the Partial Product Generator (PPG) - which generates the requisite partial sums that are then fed into the next block, the reduction tree. They can be designed somewhat independently of each other. The PPG is currently based on this⁶ paper. It is unlikely to require any changes as it satisfies our requirements quite well.

⁶<https://ieeexplore.ieee.org/document/1292369>

The core idea of the multiplier is to mask unnecessary bits with zeroes before reduction, enabling the reusability of the hardware. For example, in the given image, for 16b x 16b multiplication all cells are used. However for 8b x 8b SWP multiplication, the white diamonds are masked with zero and for 4b x 4b SWP multiplication, both the white and grey diamonds are masked with zeroes. The paper linked gives an excellent description of the same.

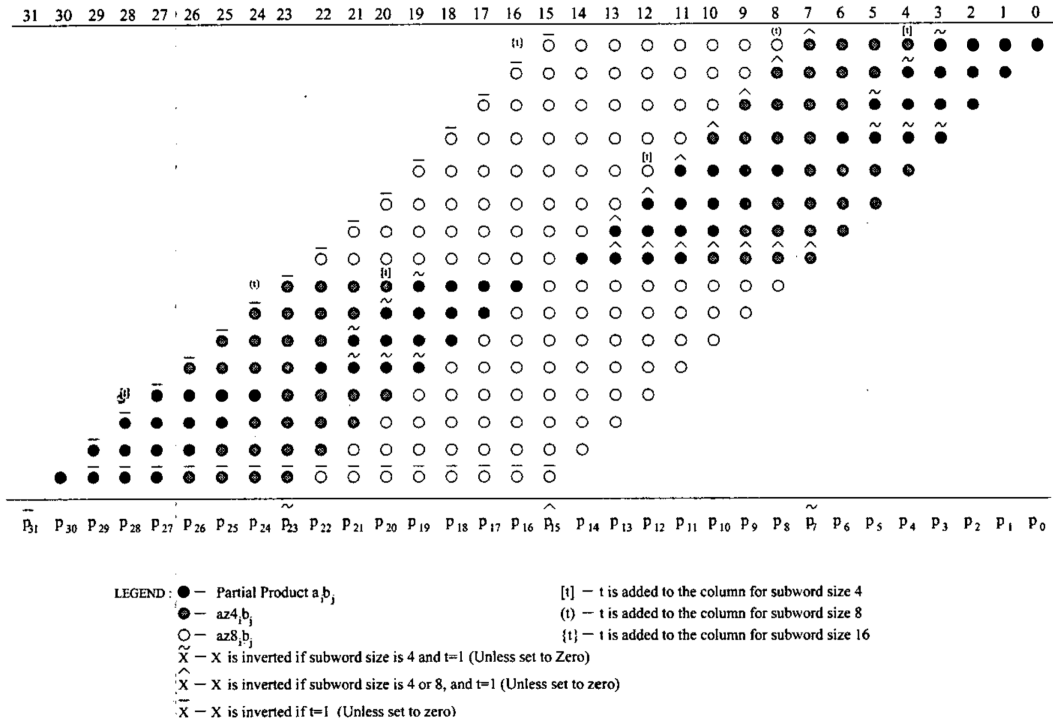


Figure 3: PPG Scheme Used

When it comes to simple reduction methods (for now only accurate schemes) - we are essentially presented two choices: the Wallace Tree⁷ and the Dadda Tree⁸. For now, we have implemented the Dadda method since we estimate that the delay is approximately the same and its on-chip area is considerably smaller.

We have not investigated the usage of more complex schemes.

⁷https://en.wikipedia.org/wiki/Dadda_multiplier

⁸https://en.wikipedia.org/wiki/Wallace_tree

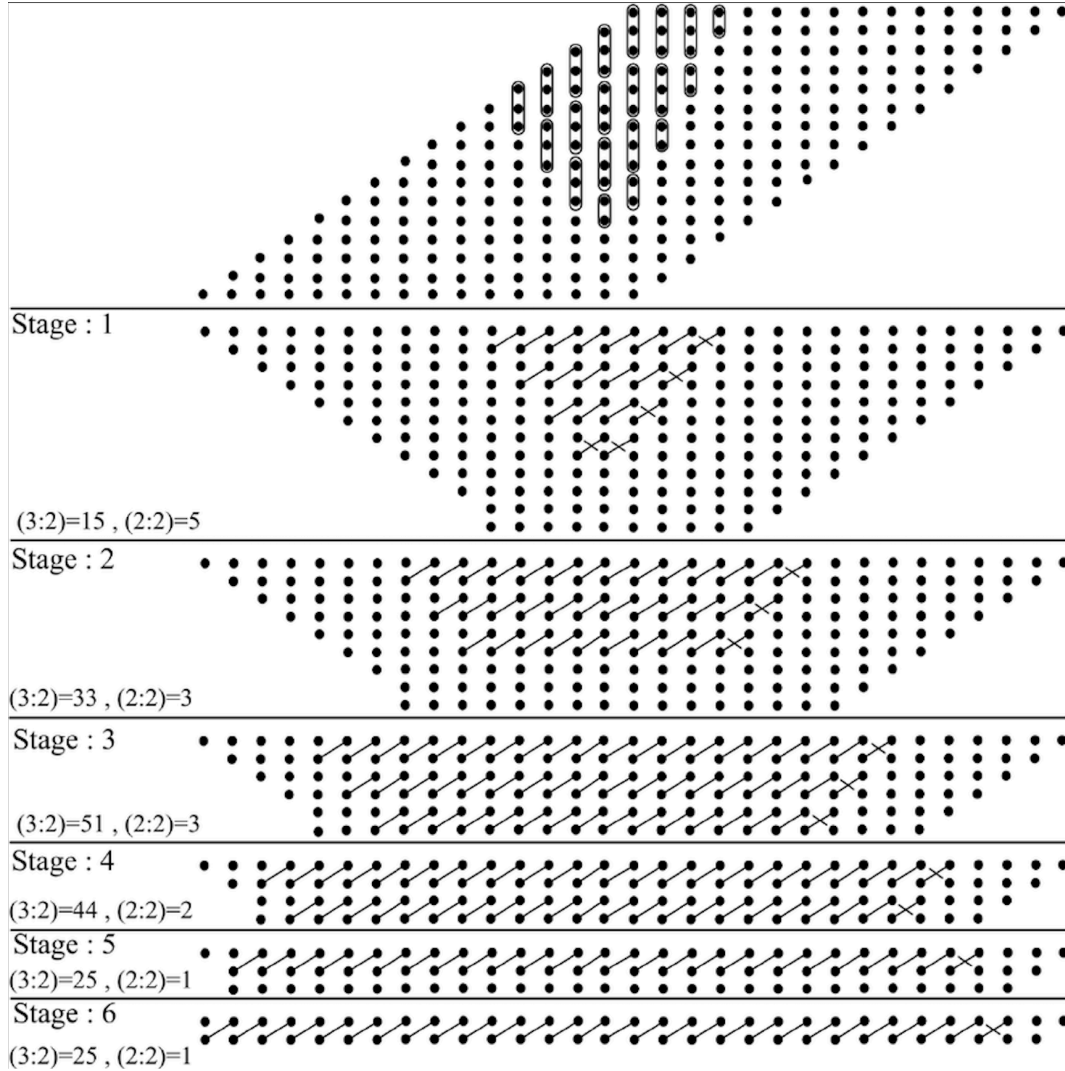


Figure 4: Dadda Reduction Tree

It may be noted that the above figure is only for 16 bits. It was too complicated to construct a 32 bit dadda multiplier, so 4 x 16 bit multipliers were used and their results reduced yet again to derive the final result.

Based on synthesis reports, the current time to beat is 5.2 ns and the benchmark area is around $175,000\mu^2$

3.2 The Adder

Constructing an SWP adder is relatively straightforward, it only requires disabling of carries at subword ends. However, for the sake of performance, we construct an SWP Carry-Lookahead Adder using the same masking principle as the multiplier. Although the adder used is actually a hybrid CLA/CSLA⁹ design, it may be thought of as a CLA for the sake of discussion.

Please note that the linked paper is translated from Chinese so it has some errors although the overall concepts are correct.

3.2.1 Multi-level CLA Design

This is recap of how CLAs work¹⁰ to illustrate the SWP adder's functionality.

For Multi-level CLAs we build on the same idea as a CLA. For example, for a 16b number, we can break it into chunks of 4 bits and initially compute C_3 , C_7 , C_{11} and C_{15} which can be fed into 4b carry generation blocks to calculate the remaining carries. For a 64b number, we can just add an additional layer connecting 4 x 16b CLAs.

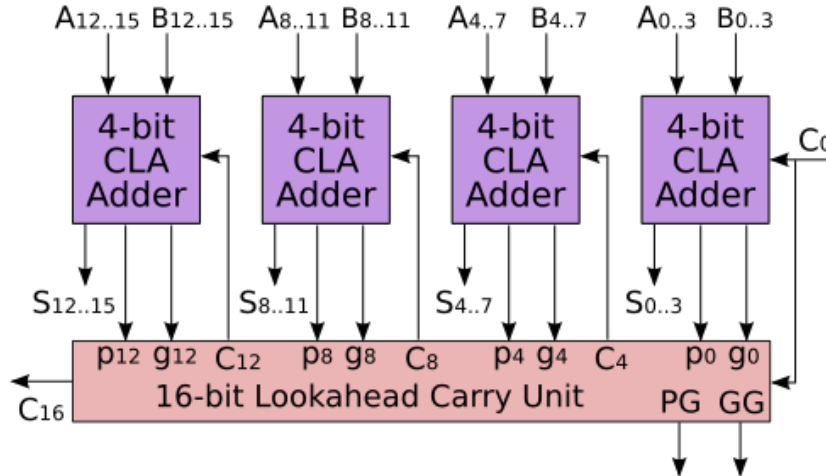


Figure 5: The image indexes carries differently

⁹<https://ieeexplore.ieee.org/document/6292045>

¹⁰https://users.encs.concordia.ca/~asim/COEN_6501/Lecture_Notes/

3.2.2 SWP CLA Design

To correctly enable SWP addition, we need to be able to effectively slice the 4b Lookahead carry units into two 2b units as we please. This is done either by ORing with a high value (for P^*) or ANDing with a low value (for G^*), as can be seen in the functions with an `'_z'` in `hybridCLA.bsv`.

The `'z'` bits are calculated based on the input control signals, similar to the multiplier.

3.3 Other Units

We have not applied the principles of SWP to any other FUs - mainly shifters and CLZ blocks as it would be unnecessarily complicated. Their configuration is decided by the compiler and not explicitly defined to allow maximum optimisation.

4 Overall Layout

The PBox effectively acts as a combined Decode and Execute Stage for P Instructions.

The Decoder takes three fields as inputs - funct7[31:25], funct3[14:12] and imm[25:20] which may act as an immediate or RS2's address based on the instruction. The opcodes are defined in 'decoder.defines' to enable the decoder to behave like a large lookup table (it is currently not optimised to generate control signals using input patterns). The decoder then generates a CtrlSgnPacket struct that contains all the necessary control signals for executions.

The individual control signals themselves are described in the Code Description section.

5 Block Descriptions

The main blocks of the datapath consist of ALUs 1 - 3, the multipliers, accumulator, decoder and some miscellaneous muxes and modules.

5.1 ALU1

ALU1 has the following three purposes:

1. Sign/Zero extension, which is currently only used for unpacking.
2. SWP crossing
3. SWP inversion as well as upper/lower inversion for subtraction

Currently, the RS1 output of ALU1 is hardwired to the RS1 input. RS2 undergoes operations serially as specified above and the unpack signal is a parallel 128 bit-wide output.

5.2 Multiplier

It is essentially just the multiplier with a decoder added to convert control signals to control bits. There are two multipliers, one for the lower words of RS1 and RS2 and one for their upper words - designated M0 and M1 respectively. Both M0 and M1 also generate bit masks that can be used for saturation if necessary.

5.3 ALU Mux

ALU Mux is currently overloaded and its operations should be assigned to different blocks for clarity. It selects the relevant bits of RS1, RS2, M0 and M1 to be fed either to writeback or ALU2. It also performs saturation for M0 and M1.

5.4 ALU2

ALU2 is the main computation and post-processing unit. It serially performs the following operations:

1. SWP Shift - Logical Right, Arithmetic Right, Left or Saturating Left Shifts. For Arithmetic Shifts, the Shifter returns 8 bits to be used for rounding. These are obtained by shifting by an amount equal to SA - 1, returning the LSB and sign-extending the remaining bits.
2. SWP Addition (alternately 'Misc.' operation) - Also used for Rounding
3. SWP Saturate/Halve - Post-processing

When any of these stages are given 'NOP' as a control signal, they return the RS1-related field as their output.

5.5 Accumulator and ALU3

Not yet implemented. In theory, Accumulator is just an adder/subtractor and ALU3 is its post-processing unit.

5.6 Writeback Control

Another overloaded Mux - it performs a wide variety of operations. It takes as input several 'metadata' bits from the adder that allow it to perform Min/Max and Eq. checks and return the relevant values. It also selects the output source.

5.7 Miscellaneous Blocks

5.7.1 Shift Amount Generator

Based on the value of SftGenCtrlSgn, it chooses the shift amount from either the lower bits of RS2 or the immediate. It generates two values: 'signed_sa'

that is the sign-extended shift amount (for KSLRAn) and the actual shift amount.

5.7.2 CLZ Blocks

These blocks perform SWP leading zero counts. CLRS (Count Leading Redundant Sign) is performed by inverting the input and using the inbuilt 'countZerosMSB' BSV function. Based on the MSB the inverted or non-inverted value is selected.

5.7.3 CMP Blocks

RS1 and RS2 are XOR'd and AND reduced. Based on the output, an 8 bit code is generated, with each bit corresponding to one byte of the output.

6 Code Description

The Code is structured as follows. The folder /topLevelDesign contains the datapath. Other folders contain FU development files. Within the /topLevelDesign files, 'topLevel.bsv' though not the top file, is effectively the highest level hardware file. 'alus.bsv' contains all blocks and sub-blocks apart from write-back control, which is contained in its own file.

Please refer the comments at the top of each file and readme.txt for more information.

The control signals are named quite directly after the operation they perform. This is a description of their functionalities.

Control Signal	Function
ExtCtrlSgn	Only used for Unpacking [ALU1]
CrsCtrlSgn	SWP Cross Control [ALU1]
InvCtrlSgn	SWP Inversion [ALU1]
MultCtrlSgn	SWP Multiplication [Multiplier]
MSatCtrlSgn	Multiplier Saturation [ALU Mux]
AluMuxCtrlSgn	ALU2 input Mux [ALU Mux]
Bool rs2Check	Alt. ALU2 Shifter Op for KSLRAn [ALU2]
SftGenCtrlSgn	Shift Amount generation source select [Top Level]
SftCtrlSgn	SWP Shifter Control [ALU2]
Alu2CtrlSgn	Main ALU2 Adder/Misc operation [ALU2]
SatCtrlSgn	ALU2 Post-processing [ALU2]
WbCtrlSgn	Writeback Control [WB Control]